

CS132S Data Structures & Algorithms

Test #2

Test 2 In Class and Test 2 Take Home are open-book (course textbook) and open-notes (your own notes from this class) and open-laptop, but closed-neighbor. You may NOT communicate with anyone about the Test before the Test 2 Take Home submission deadline, other than the Instructor. Do not forget the Academic Dishonesty Policy.

Relax, take a deep breath, and pray ☺.

Your grade will be based upon both the correctness and quality of your solutions. Don't get stuck on one portion of the test, but look all of it over first. If you cannot completely solve something, do your best, and submit whatever you are able to, so that I will have more opportunity to award any applicable partial credit.

DO NOT LEAVE THE ROOM until I verify to you that your submission for Test 2 In Class has been received successfully.

For the IN CLASS PORTION of this Test, implement solutions to ANY ONE (your choice) of the following problems. If you choose to implement solutions to more than one of these problems, you will receive normal credit for the one you do the best, plus a little bit of extra credit for each additional one of sufficient quality.

For the TAKE HOME PORTION of this Test, implement solutions to ANY TWO of the following problems.

(continued on next page)

1. Deliverable: Test2_1.py

Sample Input/Output:

>>>

RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test2_1.py

This program simulates an emergency room with limited capacity.

Event 0 is an available doctor. Other events are arriving patients.

Highest priority patient present is seen first when a doctor is free.

Lowest priority patient present is dismissed when capacity is exceeded.

Enter capacity: 3

Enter event list: 6 2 0 4 8 0 1 7 0 9 3 0 5 0 0 0 0

Enqueue patient with priority 6 (size is 1).

Enqueue patient with priority 2 (size is 2).

Dequeue patient with priority 2 (size is 1).

Enqueue patient with priority 4 (size is 2).

Enqueue patient with priority 8 (size is 3).

Dequeue patient with priority 4 (size is 2).

Enqueue patient with priority 1 (size is 3).

Enqueue patient with priority 7 (size is 4).

Dismiss patient with priority 8 (capacity exceeded).

Dequeue patient with priority 1 (size is 2).

Enqueue patient with priority 9 (size is 3).

Enqueue patient with priority 3 (size is 4).

Dismiss patient with priority 9 (capacity exceeded).

Dequeue patient with priority 3 (size is 2).

Enqueue patient with priority 5 (size is 3).

Dequeue patient with priority 5 (size is 2).

Dequeue patient with priority 6 (size is 1).

Dequeue patient with priority 7 (size is 0).

Emergency room is currently empty.

>>>

2. Deliverable: Test2_2.py

Sample Output:

```
>>> ! restart /Users/jaickemeyer/TMU/Courses/CS132S/_key/Test2_2.py
This program creates a balanced copy of a BST from an existing BST.

CS132S CS492S CS202H CS322E CS342D CS321O CS321P CS121P CS301A CS312N MA253 CS490I

Original Binary Search Tree __str__ Test:

[|K=CS132S|V=DS&A|L=CS121P|R=CS492S|P=|,
 [|K=CS121P|V=ItCP|L=|R=|P=CS132S|,
  [],
  []],
 [|K=CS492S|V=SS|L=CS202H|R=MA253|P=CS132S|,
 [|K=CS202H|V=CH|L=|R=CS322E|P=CS492S|,
  [],
  [|K=CS322E|V=SE|L=CS321O|R=CS342D|P=CS202H|,
   [|K=CS321O|V=OS|L=CS301A|R=CS321P|P=CS322E|,
    [|K=CS301A|V=CO&A|L=|R=CS312N|P=CS321O|,
     [],
     [|K=CS312N|V=NP&A|L=|R=|P=CS301A|,
      [],
      []]
    ],
    [|K=CS321P|V=PL&S|L=|R=|P=CS321O|,
     [],
     []]
  ],
  [|K=CS342D|V=DMS|L=|R=CS490I|P=CS322E|,
   [],
   [|K=CS490I|V=I|L=|R=|P=CS342D|,
    [],
    []]
  ]
],
 [|K=MA253|V=DM|L=|R=|P=CS492S|,
  [],
  []]
],
[]

Balanced Binary Search Tree __str__ Test:

[|K=CS321P|V=PL&S|L=CS301A|R=CS490I|P=|,
 [|K=CS301A|V=CO&A|L=CS132S|R=CS321O|P=CS321P|,
 [|K=CS132S|V=DS&A|L=CS121P|R=CS202H|P=CS301A|,
  [|K=CS121P|V=ItCP|L=|R=|P=CS132S|,
   [],
   []],
  [|K=CS202H|V=CH|L=|R=|P=CS132S|,
   [],
   []]
],
 [|K=CS321O|V=OS|L=CS312N|R=|P=CS301A|,
  [|K=CS312N|V=NP&A|L=|R=|P=CS321O|,
   [],
   []]
],
 [|K=CS490I|V=I|L=CS342D|R=MA253|P=CS321P|,
 [|K=CS342D|V=DMS|L=CS322E|R=|P=CS490I|,
  [|K=CS322E|V=SE|L=|R=|P=CS342D|,
   [],
   []]
],
 [|K=MA253|V=DM|L=CS492S|R=|P=CS490I|,
  [|K=CS492S|V=SS|L=|R=|P=MA253|,
   [],
   []]
],
[]

>>>
```

3. Deliverable: Test2_3.py

Sample Input/Output:

```
>>>
```

```
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test2_3.py
```

```
Welcome to Diamond Flipper!
```

```
You begin with nine coins, showing "heads", arranged in a diamond:
```

```
      H      1
     H H     2 3
    H H H   4 5 6  <= Coin Choice Options 1-9
     H H     7 8
      H      9
```

```
Choose a coin to flip it over, along with those vertically and
horizontally adjacent.
```

```
The object of the game is to end up with all coins showing "tails".
```

```
If you are stuck, choose Option 0 to show the solution.  :)
```

```
      H
     H H
    H H H
     H H
      H
```

```
Option 0: Solve; Options 1-9: Choose Node
```

```
Node 0 (000000000)
```

```
Node 0 connectedTo: [17, 24, 48, 70, 134, 194, 196, 272, 313]
```

```
Choose Option 0-9: 5
```

```
      T
     H H
    T T T
     H H
      T
```

```
Option 0: Solve; Options 1-9: Choose Node
```

```
Node 313 (100111001)
```

```
Node 313 connectedTo: [0, 41, 265, 289, 296, 383, 447, 507, 509]
```

```
Choose Option 0-9: 2
```

```
      T
     T T
    T T T
     T H
      T
```

```
Option 0: Solve; Options 1-9: Choose Node
```

```
Node 383 (101111111)
```

```
Node 383 connectedTo: [70, 111, 313, 335, 359, 366, 443, 445, 505]
```

Choose Option 0-9: 1

H
T T
T H T
T H
T

Option 0: Solve; Options 1-9: Choose Node

Node 366 (101101110)

Node 366 connectedTo: [87, 126, 296, 350, 374, 383, 426, 428, 488]

Choose Option 0-9: 0

H
T T
T H T
T H
T

Computer Chooses Option 7:

H
H T
T H T
H T
T

Computer Chooses Option 3:

H
T H
T H T
H H
T

Computer Chooses Option 1:

T
T H
T T T
H H
T

Computer Chooses Option 8:

T
T T
T T T
T T
T

Success! :)

>>>

4. Deliverable: Test2_4.py

Sample Input/Output:

>>>

```
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test2_4.py
```

This program reads in a list of courses and prerequisites, and determines a valid ordering subject to prerequisite constraints via topological sort.

The ordering prioritizes alphabetically when choosing between alternatives.

```
Reading input file "prereq.txt"...
Processing input file line->CS121P:
Processing input file line->CS132S: CS121P
Processing input file line->CS202H:
Processing input file line->MA253:
Processing input file line->CS301A: CS132S CS202H
Processing input file line->CS3210: CS132S CS202H
Processing input file line->CS312N: CS132S CS202H
Processing input file line->CS321P: CS132S
Processing input file line->CS322E: CS121P
Processing input file line->CS342D:
Processing input file line->CS490I:
Processing input file line->CS492S:
Processing input file line->CS361A:
Processing input file line->CS361M:
Processing input file line->CS362N:
Processing input file line->CS362V:
Processing input file line->MA121:
Processing input file line->CS322X:
Processing input file line->CS341W: CS121P CS342D
Processing input file line->CS351S: CS312N CS322X
Processing input file line->ACC210:
Processing input file line->MGT310:
Processing input file line->CS311C: CS312N CS342D
Processing input file line->CS392M: CS322E
Performing topological sort...
Ordered list:
['ACC210', 'CS121P', 'CS132S', 'CS202H', 'CS301A', 'CS312N', 'CS3210',
'CS321P', 'CS322E', 'CS322X', 'CS342D', 'CS311C', 'CS341W', 'CS351S',
'CS361A', 'CS361M', 'CS362N', 'CS362V', 'CS392M', 'CS490I', 'CS492S',
'MA121', 'MA253', 'MGT310']
>>>
```

~~~ *END OF CS132S TEST #2* ~~~